

SQL Code

```
1  --Exercise 1: Control Structures
2  begin
3      for l in (
4          select l.loanID, l.InterestRate, c.DOB
5          from Loans l
6          join customers c on l.CustomerID = c.CustomerId
7      )loop
8
9          if months_between(SYSDATE, l.DOB) /12 > 60 then
10             update Loans
11             set InterestRate = InterestRate - 1
12             where LoanId = l.LoanID;
13         end if;
14     end loop;
15 commit;
16 end;
17 /
18
19 alter table customers add isVIP varchar2(5);
20
21 begin
22     for c in(select customerid, balance from customers) loop
23         if c.balance > 10000 then
24             update customers
25             set isVIP = 'true'
26             where customerid = c.customerid;
27         end if;
28     end loop;
29 commit;
30 end;
31 /
32
33 begin
34     for l in(
35         select l.loanid, l.customerid, l.enddate, c.name
36         from loans l
37         join customers c on l.customerid = c.customerid
38     )loop
39         if l.enddate <= sysdate + 30 then
40             dbms_output.put_line(
41                 'reminder: loan_id' || l.loanid ||
42                 'for customer' || l.name ||
43                 'is due on' || l.enddate
44             );
45         end if;
46     end loop;
47 end;
48 /
49
50 --Exercise 2: Error Handling
```

```
51 create or replace procedure safetransferfunds(
52     p_from_acc in number,
53     p_to_acc in number,
54     p_amount in number
55 )
56 is
57 v_balance number;
58 begin
59     select balance into v_balance
60     from accounts
61     where accountid = p_from_acc;
62
63     if v_balance < p_amount then
64         raise_application_Error(-20001, 'insufficient funds');
65     end if;
66
67     update accounts
68     set balance = balance - p_amount
69     where accountid = p_from_acc;
70
71     update accounts
72     set balance = balance + p_amount
73     where accountid = p_to_acc;
74
75     commit;
76
77     exception
78         WHEN others then
79             rollback;
80 end;
81 /
82
83 create or replace procedure updatesalary(
84     p_emp_id in number,
85     p_percent in number
86 )
87 is
88 begin
89     update employees
90     set salary = salary + (salary * p_percent /100)
91     where employeeid = p_emp_id;
92
93     if sql%rowcount = 0 then
94         raise_application_error(-20002, 'Employee not found');
95     end if;
96
97     commit;
98 end;
99 /
100
101 create or replace procedure addnewcustomers(
102     p_id in number,
103     p_name in varchar2,
104     p_dob in date,
```

```
105     p_bal in number
106 )
107 is
108 begin
109     insert into customers(customerid, name, dob, balance, lastmodified)
110     values (p_id, p_name, p_dob, p_bal, sysdate);
111     commit;
112
113     exception
114         when dup_val_on_index then
115             dbms_output.put_line('error: customer id already exists');
116 end;
117 /
118
119 --Exercise 3: Stored Procedures
120 create or replace procedure processmonthlyinterest
121 is
122 begin
123     update accounts
124     set balance = balance + (balance * 0.01)
125     where accounttype = 'savings';
126     commit;
127 end;
128 /
129
130 create or replace procedure updateemployeebonus(
131     dept_name in varchar2,
132     bonus_percent in NUMBER
133 )
134 is
135 begin
136     update employees
137     set salary = salary + (salary * bonus_percent /100)
138     where department = dept_name;
139     commit;
140 end;
141 /
142
143 CREATE OR REPLACE PROCEDURE TransferFunds(
144     from_acc IN NUMBER,
145     to_acc IN NUMBER,
146     amt IN NUMBER
147 )
148 IS
149     bal NUMBER;
150 BEGIN
151     SELECT Balance INTO bal
152     FROM Accounts
153     WHERE AccountID = from_acc;
154
155     IF bal < amt THEN
156         DBMS_OUTPUT.PUT_LINE('Insufficient balance');
157     RETURN;
158     END IF;
```

```
159
160     UPDATE Accounts
161     SET Balance = Balance - amt
162     WHERE AccountID = from_acc;
163
164     UPDATE Accounts
165     SET Balance = Balance + amt
166     WHERE AccountID = to_acc;
167
168     COMMIT;
169 END;
170 /
171
172 --Exercise 4: Functions
173 create or replace function calculateAge(
174     dob in date
175 )
176 return number
177 is
178 age_years number;
179 begin
180     age_years := floor(months_between(sysdate, dob)/12);
181     return age_years;
182 end;
183 /
184
185
186 create or replace function monthlyinstallment(
187     loan_amt in number,
188     rate in number,
189     years in number
190 )
191 return number
192 is
193     total_amount number;
194     monthly_installment number;
195 begin
196     total_amount := loan_amt + (loan_amt * rate * years / 100);
197     monthly_installment := total_amount / (years * 12);
198     return monthly_installment;
199 end;
200 /
201
202 create or replace function hasSufficient(
203     acc_id number,
204     amount number
205 )
206 return boolean
207 is bal number;
208 begin
209     select balance into bal
210     from accounts
211     where accountid = acc_id;
212
```

```
213         if bal >= amount then
214             return true;
215         else
216             return false;
217         end if;
218     end;
219 /
220
221 --Exercise 5: Triggers
222 create or replace trigger updateCustomerlastmodified
223 before update on customers
224 for each row
225 begin
226     :new.lastmodified := sysdate;
227 end;
228 /
229
230 CREATE TABLE AuditLog (
231     LogID NUMBER,
232     TransactionID NUMBER,
233     AccountID NUMBER,
234     ActionDate DATE,
235     Amount NUMBER,
236     TransactionType VARCHAR2(10)
237 );
238 drop table AuditLog;
239 CREATE OR REPLACE TRIGGER LogTransaction
240 AFTER INSERT ON Transactions
241 FOR EACH ROW
242 BEGIN
243     INSERT INTO AuditLog(LogID, TransactionID, AccountID, ActionDate, Amount,
244 TransactionType)
245     VALUES(:NEW.TransactionID, :NEW.TransactionID, :NEW.AccountID, SYSDATE, :NEW.Amount,
246 :NEW.TransactionType);
247 END;
248 /
249
250 create or replace trigger checkTrasactionRules
251 before insert on transactions
252 for each row
253 declare
254     acc_balance number;
255 begin
256     if :new.transactiontype = 'deposit' and :new.amount <= 0 then
257         raise_application_error(-20001, 'deposit amount must be positive');
258     end if;
259
260     if :new.transactiontype = 'withdrawal' then
261         select balance into acc_balance
262         from accounts
263         where accountid = :new.accountid;
264
265         if :new.amount > acc_balance then
266             raise_application_error(-20002, 'efficient balance fro withdrawal');
```

```
265         end if;
266     end if;
267 end;
268 /
269
270
271 --Exercie 6: cursors
272 DECLARE
273     CURSOR GenerateMonthlyStatements IS
274         SELECT c.CustomerID, c.Name, t.TransactionID, t.TransactionDate, t.Amount,
275                t.TransactionType
276         FROM Customers c
277         JOIN Accounts a ON c.CustomerID = a.CustomerID
278         JOIN Transactions t ON a.AccountID = t.AccountID
279         WHERE EXTRACT(MONTH FROM t.TransactionDate) = EXTRACT(MONTH FROM SYSDATE)
280               AND EXTRACT(YEAR FROM t.TransactionDate) = EXTRACT(YEAR FROM SYSDATE);
281
282     v_custid Customers.CustomerID%TYPE;
283     v_name Customers.Name%TYPE;
284     v_tid Transactions.TransactionID%TYPE;
285     v_date Transactions.TransactionDate%TYPE;
286     v_amt Transactions.Amount%TYPE;
287     v_type Transactions.TransactionType%TYPE;
288
289 BEGIN
290     OPEN GenerateMonthlyStatements;
291
292     LOOP
293         FETCH GenerateMonthlyStatements
294         INTO v_custid, v_name, v_tid, v_date, v_amt, v_type;
295
296         EXIT WHEN GenerateMonthlyStatements%NOTFOUND;
297
298         DBMS_OUTPUT.PUT_LINE('Customer ID: ' || v_custid ||
299                               ', Name: ' || v_name ||
300                               ', Transaction ID: ' || v_tid ||
301                               ', Date: ' || v_date ||
302                               ', Amount: ' || v_amt ||
303                               ', Type: ' || v_type);
304     END LOOP;
305
306     CLOSE GenerateMonthlyStatements;
307 END;
308 /
309
310 DECLARE
311     CURSOR ApplyAnnualFee IS
312         SELECT AccountID, Balance
313         FROM Accounts;
314
315     v_accid Accounts.AccountID%TYPE;
316     v_balance Accounts.Balance%TYPE;
317     annual_fee NUMBER := 500;
```

```
318 BEGIN
319     OPEN ApplyAnnualFee;
320
321     LOOP
322         FETCH ApplyAnnualFee INTO v_accid, v_balance;
323         EXIT WHEN ApplyAnnualFee%NOTFOUND;
324
325         UPDATE Accounts
326         SET Balance = Balance - annual_fee
327         WHERE AccountID = v_accid;
328     END LOOP;
329
330     CLOSE ApplyAnnualFee;
331     COMMIT;
332 END;
333 /
334
335 DECLARE
336     CURSOR UpdateLoanInterestRates IS
337         SELECT LoanID, LoanAmount, InterestRate
338         FROM Loans;
339
340     v_loanid Loans.LoanID%TYPE;
341     v_amount Loans.LoanAmount%TYPE;
342     v_rate Loans.InterestRate%TYPE;
343
344 BEGIN
345     OPEN UpdateLoanInterestRates;
346
347     LOOP
348         FETCH UpdateLoanInterestRates INTO v_loanid, v_amount, v_rate;
349         EXIT WHEN UpdateLoanInterestRates%NOTFOUND;
350
351         IF v_amount > 500000 THEN
352             UPDATE Loans
353             SET InterestRate = InterestRate + 0.5
354             WHERE LoanID = v_loanid;
355         ELSE
356             UPDATE Loans
357             SET InterestRate = InterestRate + 0.2
358             WHERE LoanID = v_loanid;
359         END IF;
360     END LOOP;
361
362     CLOSE UpdateLoanInterestRates;
363     COMMIT;
364 END;
365 /
366
367 --Exercie 7: packages
368 CREATE OR REPLACE PACKAGE CustomerManagement AS
369     PROCEDURE AddCustomer(
370         p_id NUMBER,
371         p_name VARCHAR2,
```

```
372     p_dob DATE,
373     p_balance NUMBER
374 );
375
376 PROCEDURE UpdateCustomer(
377     p_id NUMBER,
378     p_name VARCHAR2,
379     p_dob DATE,
380     p_balance NUMBER
381 );
382
383 FUNCTION GetBalance(
384     p_id NUMBER
385 ) RETURN NUMBER;
386 END CustomerManagement;
387 /
388
389 CREATE OR REPLACE PACKAGE BODY CustomerManagement AS
390
391     PROCEDURE AddCustomer(
392         p_id NUMBER,
393         p_name VARCHAR2,
394         p_dob DATE,
395         p_balance NUMBER
396     )
397     IS
398     BEGIN
399         INSERT INTO Customers(CustomerID, Name, DOB, Balance, LastModified)
400         VALUES(p_id, p_name, p_dob, p_balance, SYSDATE);
401     END;
402
403     PROCEDURE UpdateCustomer(
404         p_id NUMBER,
405         p_name VARCHAR2,
406         p_dob DATE,
407         p_balance NUMBER
408     )
409     IS
410     BEGIN
411         UPDATE Customers
412         SET Name = p_name,
413             DOB = p_dob,
414             Balance = p_balance,
415             LastModified = SYSDATE
416         WHERE CustomerID = p_id;
417     END;
418
419     FUNCTION GetBalance(
420         p_id NUMBER
421     ) RETURN NUMBER
422     IS
423         v_balance NUMBER;
424     BEGIN
425         SELECT Balance INTO v_balance
```



```
426         FROM Customers
427         WHERE CustomerID = p_id;
428
429     RETURN v_balance;
430 END;
431
432 END CustomerManagement;
433 /
434
435 CREATE OR REPLACE PACKAGE AccountOperations AS
436     PROCEDURE OpenAccount(
437         p_accid NUMBER,
438         p_custid NUMBER,
439         p_type VARCHAR2,
440         p_balance NUMBER
441     );
442
443     PROCEDURE CloseAccount(
444         p_accid NUMBER
445     );
446
447     FUNCTION GetTotalBalance(
448         p_custid NUMBER
449     ) RETURN NUMBER;
450 END AccountOperations;
451 /
452
453 CREATE OR REPLACE PACKAGE BODY AccountOperations AS
454
455     PROCEDURE OpenAccount(
456         p_accid NUMBER,
457         p_custid NUMBER,
458         p_type VARCHAR2,
459         p_balance NUMBER
460     )
461     IS
462     BEGIN
463         INSERT INTO Accounts(AccountID, CustomerID, AccountType, Balance, LastModified)
464         VALUES(p_accid, p_custid, p_type, p_balance, SYSDATE);
465     END;
466
467     PROCEDURE CloseAccount(
468         p_accid NUMBER
469     )
470     IS
471     BEGIN
472         DELETE FROM Accounts
473         WHERE AccountID = p_accid;
474     END;
475
476     FUNCTION GetTotalBalance(
477         p_custid NUMBER
478     ) RETURN NUMBER
479     IS
```

```
480     v_total NUMBER;  
481 BEGIN  
482     SELECT SUM(Balance) INTO v_total  
483     FROM Accounts  
484     WHERE CustomerID = p_custid;  
485  
486     RETURN v_total;  
487 END;  
488  
489 END AccountOperations;  
490 /
```